Studio Nx

Backend(FastAPI/Python) 22 — POST /api/v1/auth/signup

2026-10-12

Backend(FastAPI/Python) 22 — 라우터 `POST /api/v1/auth/signup`

항목	내용
파일	`studionx/backend/app/api/v1/auth.py` (함수 이름 `signup`)
작업	P1-04 회원가입: 가입 API (PRD 01 FR-101, §7)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	21 `signup()` · 23 `GET /auth/check-email`

1. 하는 일

브라우저가 보낸 가입 요청을 받아 21번 `auth_service.signup()`에 넘기고, 결과를 **응답 모양**으로 바꿔서 돌려줍니다.

```
POST /api/v1/auth/signup
{"email": "Route.User@Example.com", "nickname": "route_user", "password": "Abcdef1!",
 "privacy_agreed": true, "privacy_version": "2026-10-05"}

→ 201 Created
{"user_id": 92, "email": "route.user@example.com", "nickname": "route_user"}
```

2. 언제 쓰나

join.html의 **Sign Up** 버튼을 누르면 `join.js`(P1-07)가 이 주소로 요청을 보냅니다. 201을 받으면 화면은 `home.html#login`으로 옮겨 로그인 모달을 엽니다 (가입한 뒤 자동으로 로그인시키지는 않습니다).

라우터와 서비스의 차이

층	파일	하는 일
라우터 (이 함수)	`api/v1/auth.py`	주소, 방식(`POST`), 응답 상태(201), 응답 모양을 정합니다. HTTP를 아는 층입니다
서비스 (21)	`services/auth_service.py`	실제 검사와 저장을 합니다. HTTP를 모르기 때문에 시험이나 다른 곳에서도 그대로 부를 수 있습니다

요청 하나가 지나가는 길

순서	처리하는 쪽	일어나는 일
①	FastAPI	본문 JSON을 `SignupRequest`와 비교합니다. 칸이 빠졌거나 모양이 틀리면 **라우터가 실행되기 전에** 422로 응답합니다 (02번 처리기)
②	FastAPI	`db: Db`를 보고 `get_db()`로 DB 세션을 엽니다
③	**이 함수**	`auth_service.signup(db, data)`를 부릅니다
④	21	검사하고 저장합니다. 실패하면 `AppError`를 던지고, 01번 처리기가 409나 422로 응답합니다
⑤	**이 함수**	`User`를 `SignupResponse`(번호, 이메일, 닉네임만)로 바꿉니다

순서	처리하는 쪽	일어나는 일
⑥	FastAPI	201과 JSON으로 응답합니다. 요청이 끝나면 `get_db()`가 세션을 닫습니다

결과	상태	예
가입됨	201	`{"user_id": 92, …}`
칸이 빠지거나 모양이 틀림	422	`details: {"privacy_agreed": "Field required"}`
비밀번호 규칙 위반, 동의하지 않음	422	`details: {"password": ["upper", …]}`
이메일이나 닉네임이 사용 중, 탈퇴한 지 3일이 안 됨	409	`code: "EMAIL_TAKEN"` 등

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import APIRouter, Cookie, Query, Response, status

from app.api.deps import Db
from app.schemas.auth import (
    NICKNAME_MAX,
    AvailabilityResponse,
    LoginRequest,
    SignupRequest,
    SignupResponse,
    TokenResponse,
)
from app.schemas.common import ErrorResponse
from app.services import auth_service
```

`auth.py` 맨 위의 import입니다 (P1-05 로그인에서 쓰는 것까지 함께 있습니다). **`from app.services import auth_service`를 이번에 추가했습니다.** 이 함수는 `APIRouter`(→ `router`), `status`, `Db`, `SignupRequest`, `SignupResponse`, `ErrorResponse`, `auth_service`를 사용합니다.

import	쓰는 곳
`router = APIRouter()`	`@router.post(…)`로 이 파일의 주소들을 묶습니다. `api/v1/router.py`가 이것을 `/auth` 아래에 붙입니다
`status.HTTP_201_CREATED`	성공 상태 201
`Db`	DB 세션 (P1-03 `deps.py`)
`SignupRequest`, `SignupResponse`	요청과 응답의 모양
`ErrorResponse`	API 문서(`/docs`)에 409와 422 응답 모양을 보여 줍니다
`auth_service`	21번의 `signup()`을 `auth_service.signup`으로 부릅니다

함수

```
@router.post(
    "/signup",
```

```
    response_model=SignupResponse,
    status_code=status.HTTP_201_CREATED,
    responses={409: {"model": ErrorResponse}, 422: {"model": ErrorResponse}},
)
def signup(data: SignupRequest, db: Db) -> SignupResponse:
    user = auth_service.signup(db, data)
    return SignupResponse(user_id=user.user_id, email=user.email, nickname=user.nickname)
```

위의 `@router.post(…)` 부분은 템플릿에 있었고, 함수 몸통 두 줄을 이번에 채웠습니다.

문법 풀이

import 문법	뜻
`from app.services import auth_service`	**모듈(파일) 전체**를 이름 하나로 가져와서 `auth_service.signup`처럼 씁니다. 이 파일의 라우터 함수 이름도 `signup`이라서, 서비스 함수를 `from … import signup`으로 가져오면 이름이 겹칩니다

식	뜻
`@router.post("/signup", …)`	데코레이터입니다. 아래 함수를 `POST /signup` 요청에 연결합니다
`response_model=SignupResponse`	응답을 이 모양으로 **걸러서** 보냅니다. 다른 칸이 있어도 나가지 않습니다
`status_code=201`	성공하면 200이 아니라 201(만들어짐)로 응답합니다
`responses={409: …, 422: …}`	응답 설명을 API 문서에만 추가합니다 (동작에는 영향이 없습니다)
`data: SignupRequest`	요청 본문 JSON을 이 모양으로 읽어서 넣어 줍니다 (①)
`db: Db`	DB 세션을 넣어 줍니다 (②)

4. 흐름

1. FastAPI가 본문을 검사하고 DB 세션을 엽니다.
2. 21번 `signup()`으로 가입시킵니다.
3. 번호, 이메일, 닉네임만 담아서 201로 응답합니다.

5. 왜 이렇게

선택	이유
라우터는 두 줄	검사와 저장은 서비스(21)에 둡니다. 라우터가 얇으면 HTTP 없이도 서비스를 시험할 수 있습니다
`SignupResponse`로 바꿔서 응답	`User`를 그대로 내보내면 `password_hash` 같은 칸이 섞일 위험이 있습니다. 응답 모양을 정해 두면 그 칸만 나갑니다 (PRD 01 §5)
201로 응답	새로 만들었다는 뜻의 REST 관례입니다. 화면은 201을 보고 "가입 성공"을 압니다
오류는 여기서 잡지 않음	21번이 던진 `AppError`를 01번 처리기가 받아 모양을 맞춥니다. 라우터마다 `try`를 쓰지 않아도 됩니다
가입한 뒤 토큰을 주지 않음	로그인은 P1-05가 맡는 일입니다. 가입과 로그인을 나눠 두면 각각이 단순해집니다

6. 시험 (`tests/test_auth_routes.py`에 4개를 추가해서 모두 6개, 실제 앱과 실제 MySQL로 실행하고 rollback)

시험에서는 앱의 `get_db`를 시험용 세션으로 바꿔 끼우고(`app.dependency_overrides`), 실제 주소로 요청했습니다.

시험	확인한 것
가입	201이고, 응답 칸이 정확히 `user_id`, `email`, `nickname`이며(비밀번호와 해시 없음), 이메일은 소문자이고, DB에 저장되어 있습니다
같은 이메일로 다시 가입	두 번째는 409 `EMAIL_TAKEN`입니다
비밀번호 규칙	422이고, `details.password = ["upper", "number", "special"]`입니다
칸이 빠짐	422이고, `details = {"privacy_agreed": "Field required"}`입니다 (02번 처리기)

전체 시험 149개가 통과했고, `ruff` 검사도 통과했습니다. 시험이 끝난 뒤 남은 행은 0개입니다.